

AI Inference Beyond CUDA

Building a heterogeneous runtime for GPUs, NPUs and CPUs

42 Berlin AI Club · Alumni Talk

One Token, End to End

How are you doing [3923, 527, 499, 3815] example IDs



“How” · “ are” · “ you” · “ doing” → 4 tokens

The loop repeats once for every generated token

Load model

Allocate
memory

Execute graph

Schedule work

Keep KV state

Model runtime

Token IDs to logits

Serving engine

Scheduling, batching, KV cache and sampling

What Lives in a Model Repository? In Huggingface?

MODEL DEFINITION

config.json

layers, hidden size, attention heads

LEARNED PARAMETERS

model-00001-of-00004 .safetensors

billions of trained numerical values

TEXT INTERFACE

tokenizer.json tokenizer_config.json

text, tokens and special symbols

GENERATION POLICY

generation_config.json

sampling parameters and stopping behavior

**These files describe a model.
They are not a program compiled
for your device.**

<https://huggingface.co/Qwen/Qwen3.8-27B/tree/main>



 VoyagerXHF	Update README.md	1d4bf0f	VERIFIED	24 days ago	
 .gitattributes	 Safe	1.57 kB		Upload folder using huggingface_hub	25 days ago
 LICENSE	 Safe	11.5 kB		Add LICENSE	24 days ago
 README.md	 Safe	65 kB		Update README.md	24 days ago
 chat_template.jinja	 Safe	8.95 kB		Upload chat_template.jinja with huggingface_hub	25 days ago
 config.json	 Safe	4.31 kB		Upload folder using huggingface_hub	25 days ago
 crc32.txt	 Safe	238 Bytes		Upload folder using huggingface_hub	25 days ago
 generation_config.json	 Safe	202 Bytes		Upload generation_config.json with huggingface_hub	25 days ago
 merges.txt	 Safe	3.35 MB		Upload folder using huggingface_hub	25 days ago
 model-00001-of-00018.safetensors	 Safe	3.97 GB	 xet 	Upload folder using huggingface_hub	25 days ago
 model-00002-of-00018.safetensors	 Safe	3.04 GB	 xet 	Upload folder using huggingface_hub	25 days ago
 model-00003-of-00018.safetensors	 Safe	2.54 GB	 xet 	Upload folder using huggingface_hub	25 days ago
 model-00004-of-00018.safetensors	 Safe	3.99 GB	 xet 	Upload folder using huggingface_hub	25 days ago
 model-00005-of-00018.safetensors	 Safe	2.1 GB	 xet 	Upload folder using huggingface_hub	25 days ago
 model-00006-of-00018.safetensors	 Safe	3.98 GB	 xet 	Upload folder using huggingface_hub	25 days ago
 model-00007-of-00018.safetensors	 Safe	2.11 GB	 xet 	Upload folder using huggingface_hub	25 days ago

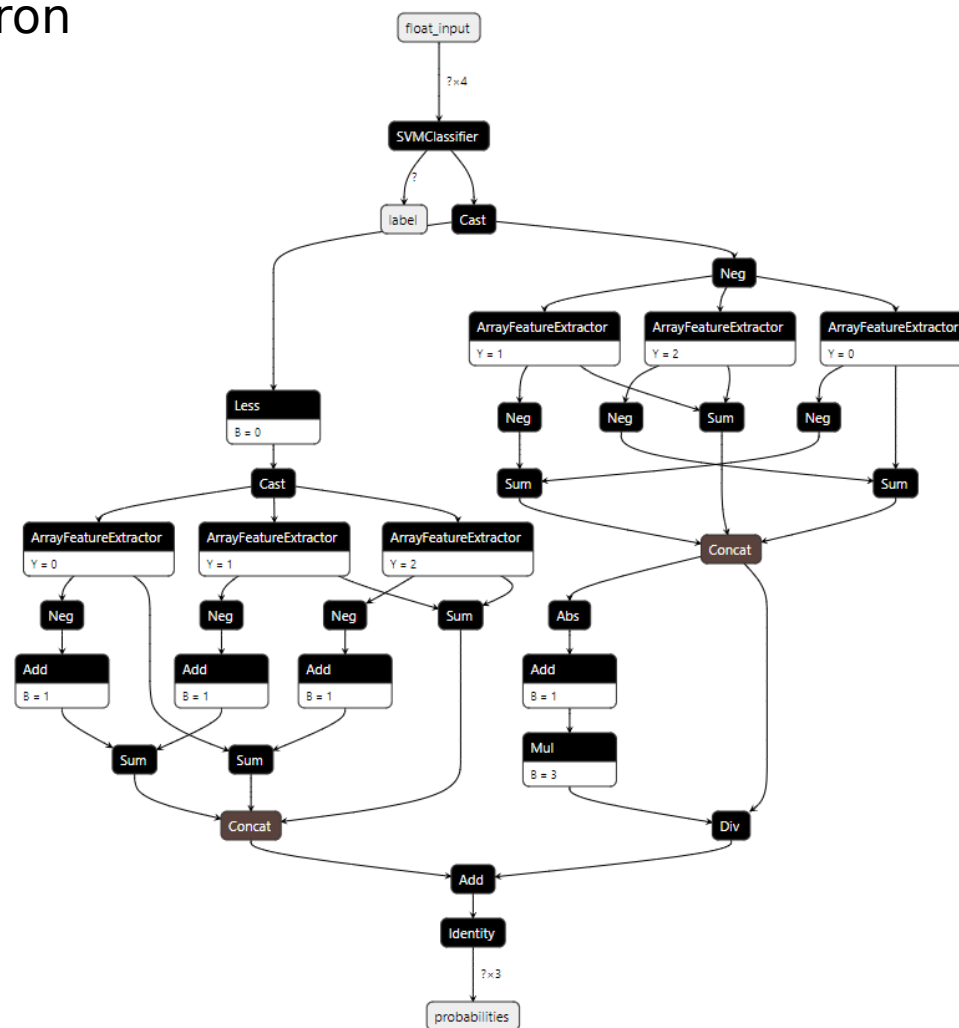
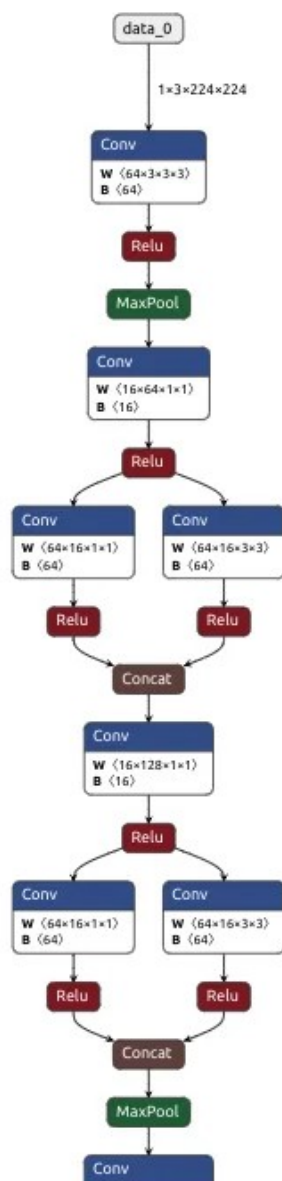

```
P±^@@^@^@^@^@{"__metadata__":{"format":"pt"},"model.language_model.layers.0.input_layernorm.weight":{"dtype":"BF16","shape":[5120],"data_offsets":[0,10240]},"model.language_model.layers.0.linear_attn.A_log":{"dtype":"BF16","shape":[48],"data_offsets":[10240,10336]},"model.language_model.layers.0.linear_attn.conv1d.weight":{"dtype":"BF16","shape":[10240,1,4],"data_offsets":[10336,92256]},"model.language_model.layers.0.linear_attn.dt_bias":{"dtype":"BF16","shape":[48],"data_offsets":[92256,92352]},"model.language_model.layers.0.linear_attn.in_proj_a.weight":{"dtype":"BF16","shape":[48,5120],"data_offsets":[92352,583872]},"model.language_model.layers.0.linear_attn.in_proj_b.weight":{"dtype":"BF16","shape":[48,5120],"data_offsets":[583872,1075392]},"model.language_model.layers.0.linear_attn.in_proj_qkv.weight":{"dtype":"BF16","shape":[10240,5120],"data_offsets":[1075392,105932992]},"model.language_model.layers.0.linear_attn.in_proj_z.weight":{"dtype":"BF16","shape":[6144,5120],"data_offsets":[105932992,168847552]},"model.language_model.layers.0.linear_attn.norm.weight":{"dtype":"BF16","shape":[128],"data_offsets":[168847552,168847808]},"model.language_model.layers.0.linear_attn.out_proj.weight":{"dtype":"BF16","shape":[5120,6144],"data_offsets":[168847808,231762368]},"model.language_model.layers.0.mlp.down_proj.weight":{"dtype":"BF16","shape":[5120,17408],"data_offsets":[231762368,410020288]},"model.language_model.layers.0.mlp.gate_proj.weight":{"dtype":"BF16","shape":[17408,5120],"data_offsets":[410020288,588278208]},"model.language_model.layers.0.mlp.up_proj.weight":{"dtype":"BF16","shape":[17408,5120],"data_offsets":[588278208,766536128]},"model.language_model.layers.0.post_attention_layernorm.weight":{"dtype":"BF16","shape":[5120],"data_offsets":[766536128,766546368]},"model.language_model.layers.1.input_layernorm.weight":{"dtype":"BF16","shape":[5120],"data_offsets":[766546368,766556608]},"model.language_model.layers.1.linear_attn.A_log":{"dtype":"BF16","shape":[48],"data_offsets":[766556608,766556704]},"model.language_model.layers.1.linear_attn.conv1d.weight":{"dtype":"BF16","shape":[10240,1,4],"data_offsets":[766556704,766638624]},"model.language_model.layers.1.linear_attn.dt_bias":{"dtype":"BF16","shape":[48],"data_offsets":[766638624,766638720]},"model.language_model.layers.1.linear_attn.in_proj_a.weight":{"dtype":"BF16","shape":[48,5120],"data_offsets":[766638720,767130240]},"model.language_model.layers.1.linear_attn.in_proj_b.weight":{"dtype":"BF16","shape":[48,5120],"data_offsets":[767130240,767621760]},"model.language_model.layers.1.linear_attn.in_proj_qkv.weight":{"dtype":"BF16","shape":[10240,5120],"data_offsets":[767621760,872479360]},"model.language_model.layers.1.linear_attn.in_proj_z.weight":{"dtype":"BF16","shape":[6144,5120],"data_offsets":[872479360,935393920]},"model.language_model.layers.1.linear_attn.norm.weight":{"dtype":"BF16","shape":[128],"data_offsets":[935393920,935394176]},"model.language_model.layers.1.linear_attn.out_proj.weight":{"dtype":"BF16","shape":[5120,6144],"data_offsets":[935394176,998308736]},"model.language_model.layers.1.mlp.down_proj.weight":{"dtype":"BF16","shape":[5120,17408],"data_offsets":[998308736,1176566656]},"model.language_model.layers.1.mlp.gate_proj.weight":{"dtype":"BF16","shape":[17408,5120],"data_offsets":[1176566656,1354824576]},"model.language_model.layers.1.mlp.up_proj.weight":{"dtype":"BF16","shape":[17408,5120],"data_offsets":[1354824576,1533082496]},"model-00001-of-00018.safetensors" [converted] 9922368L, 5949498007B
```


This file is stored with [Xet](#) . It is too big to display, but you can still [download](#) it.

Xet Pointer Details (🔗 [Raw pointer file](#))
Xet hash: 6026d6bb4baa574f54205e2894816127bcf444bb4f727aa76a4fd3c90e810140
Size of remote file: 3.97 GB · **SHA256:** ba0ce20aae489ad196733da5064bcd159a1fe84f53336648196e1ebb7751b1c
Xet efficiently stores Large Files inside Git, intelligently splitting files into unique chunks and accelerating uploads and downloads. [More info](#).

Metadata	Value	
format	pt	
Tensors	Shape	Precision
model.language_model.layers (5) ▾		
model.language_model.layers.0 (4) ▾		
model.language_model.layers.0.input_layernorm.weight	[5 120]	BF16
model.language_model.layers.0.linear_attn (9) ▾		
model.language_model.layers.0.linear_attn.A_log	[48]	BF16
model.language_model.layers.0.linear_attn.conv1d.weight	[10 240, 1, 4]	BF16
model.language_model.layers.0.linear_attn.dt_bias	[48]	BF16
model.language_model.layers.0.linear_attn.in_proj_a.weight	[48, 5 120]	BF16
model.language_model.layers.0.linear_attn.in_proj_b.weight	[48, 5 120]	BF16
model.language_model.layers.0.linear_attn.in_proj_qkv.weight	[10 240, 5 120]	BF16
model.language_model.layers.0.linear_attn.in_proj_z.weight	[6 144, 5 120]	BF16
model.language_model.layers.0.linear_attn.norm.weight	[128]	BF16
model.language_model.layers.0.linear_attn.out_proj.weight	[5 120, 6 144]	BF16
model.language_model.layers.0.mlp (3) ▾		
model.language_model.layers.0.mlp.down_proj.weight	[5 120, 17 408]	BF16
model.language_model.layers.0.mlp.gate_proj.weight	[17 408, 5 120]	BF16
model.language_model.layers.0.mlp.up_proj.weight	[17 408, 5 120]	BF16
model.language_model.layers.0.post_attention_layernorm.weight	[5 120]	BF16
model.language_model.layers.1 (4) >		
model.language_model.layers.2 (4) >		
model.language_model.layers.3 (4) >		
model.language_model.layers.4 (3) ▾		

https://github.com/lutzroeder/netron



MODEL PROPERTIES

format: ONNX v7
producer: skl2onnx 1.15.0
domain: ai.onnx
imports: ai.onnx v11
ai.onnx.ml v1
graph: d636bb672f43437486002eb7dea6857a

INPUTS

float_input: name: float_input
tensor: f1oat32[?,4]

OUTPUTS

label: name: label
tensor: int64[?]
probabilities: name: probabilities
tensor: f1oat32[?,3]

<https://share.google/I4O8epHvpqc8U4mpb>
<https://share.google/0zgovbt93v8Yr8ACp>

Qwen

Qwen3.8-27B

like

14.2k

Follow

Qwen

103k

Image-Text-to-Text

Transformers

Safetensors

qwen3_5

conversational

Eval Results

Lice

Model card

Files and versions

xet

Community

188

main

Qwen3.8-27B

55.6 GB

VoyagerXHF

Update README.md

1d4bf0f

VERIFIED

.gitattributes

Safe

1.

LICENSE

Safe

11

README.md

Safe

chat_template.jinja

Safe

8.

config.json

Safe

4.

crc32.txt

Safe

238

generation_config.json

Safe

202

merges.txt

Safe

3.

model-00001-of-00018.safetensors

Safe

3.97 GB

model-00002-of-00018.safetensors

Safe

3.04 GB

model-00003-of-00018.safetensors

Safe

2.54 GB

model-00004-of-00018.safetensors

Safe

3.99 GB

model-00005-of-00018.safetensors

Safe

2.1 GB

model-00006-of-00018.safetensors

Safe

3.98 GB

model-00007-of-00018.safetensors

Safe

2.11 GB

2 contributors

History: 16

huggingface_hub

huggingface_hub

huggingface_hub

huggingface_hub

config.json with huggingface_hub

25 days ago

Upload folder using huggingface_hub

25 days ago

Deploy

Copy to bucket

NEW

Use this model

Libraries

Transformers

Inference

Inference Providers

HuggingChat

Notebooks

Google Colab

Kaggle

Local Apps

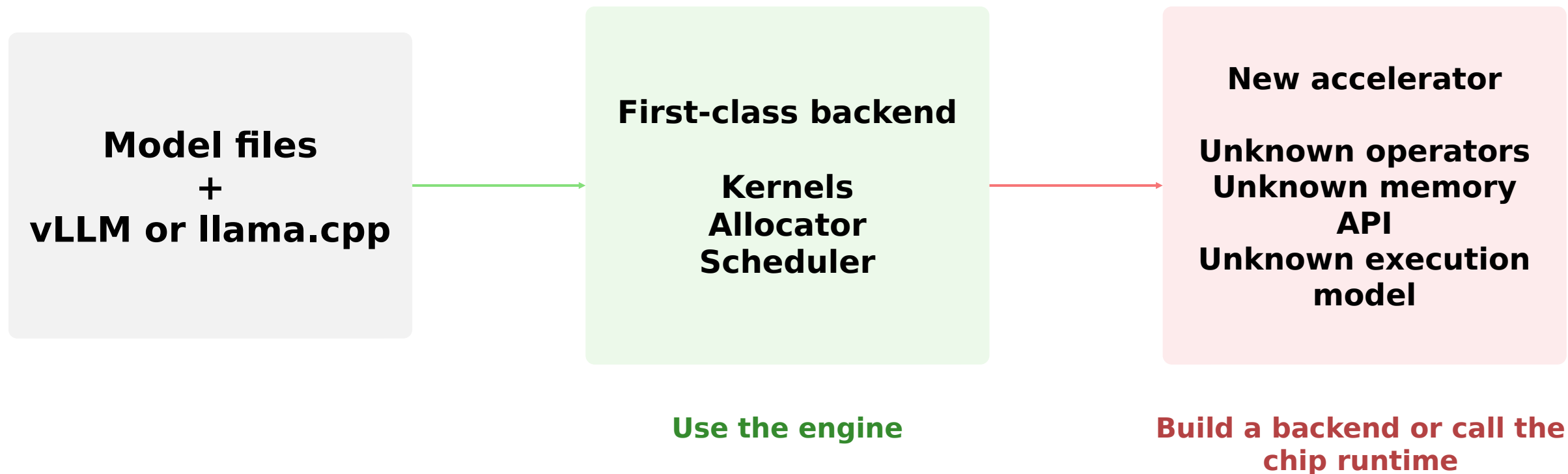
vLLM

SGLang

View all local apps

Settings

Why isn't vLLM or llama.cpp enough?



An engine can only optimize hardware that its backend understands.

One Model, Different Software Stacks

HARDWARE

NVIDIA GPU

AMD GPU

Apple Silicon

Custom NPU

RUNTIME

CUDA Runtime

ROCm and HIP

Metal and Core ML

Vendor compiler

KERNEL OR COMPILER STACK

cuBLAS and custom kernels

rocBLAS and custom kernels

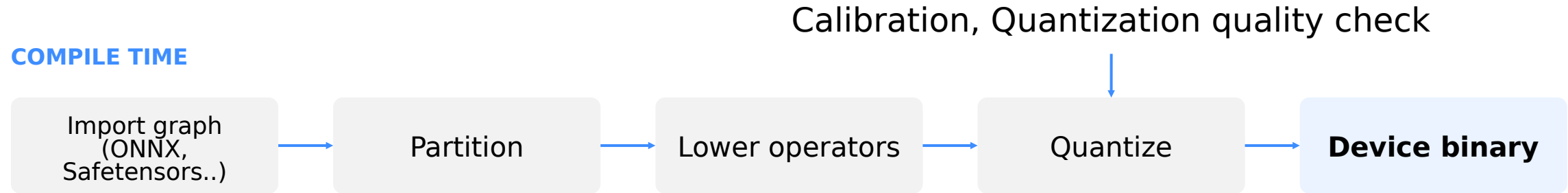
Apple device backends

Vendor runtime API

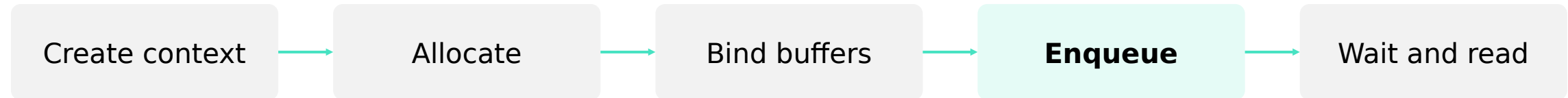
Portability ends where the backend contract ends

The Hardware Backend

COMPILE TIME



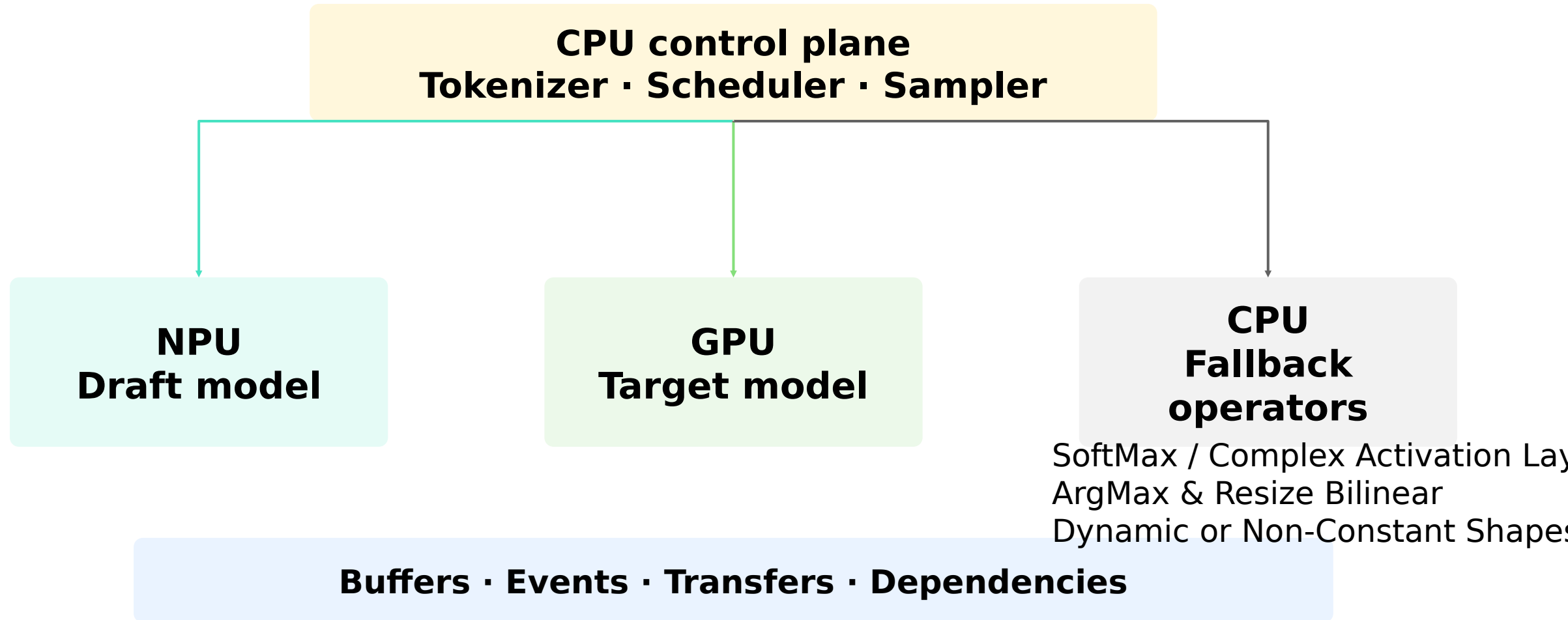
RUNTIME



```
context = create_context(device)
graph = load_binary(context, model_bin)
bind_buffer(graph, INPUT, input)
event = enqueue(graph, queue)
wait(event)
```

A Heterogeneous Inference Engine

PLACEMENT OBJECTIVE: END-TO-END LATENCY, NOT ISOLATED DEVICE TIME



Compilation, Quantization and Calibration

WEIGHT STORAGE FOR A 7B MODEL

FP16

≈ 14 GB

INT8

≈ 7 GB

INT4

≈ 3.5 GB

Weights only. Metadata and runtime buffers are excluded.

STATIC ACTIVATION QUANTIZATION

Representative inputs



Measure activation ranges



Fix scale and zero point

Weight-only INT4 may not require activation calibration

LLM Inference Uses Multiple Graphs

COMPILED ARTIFACTS

prefill_bs1_seq128.bin

prefill_bs1_seq512.bin

decode_bs1.bin

decode_bs4.bin

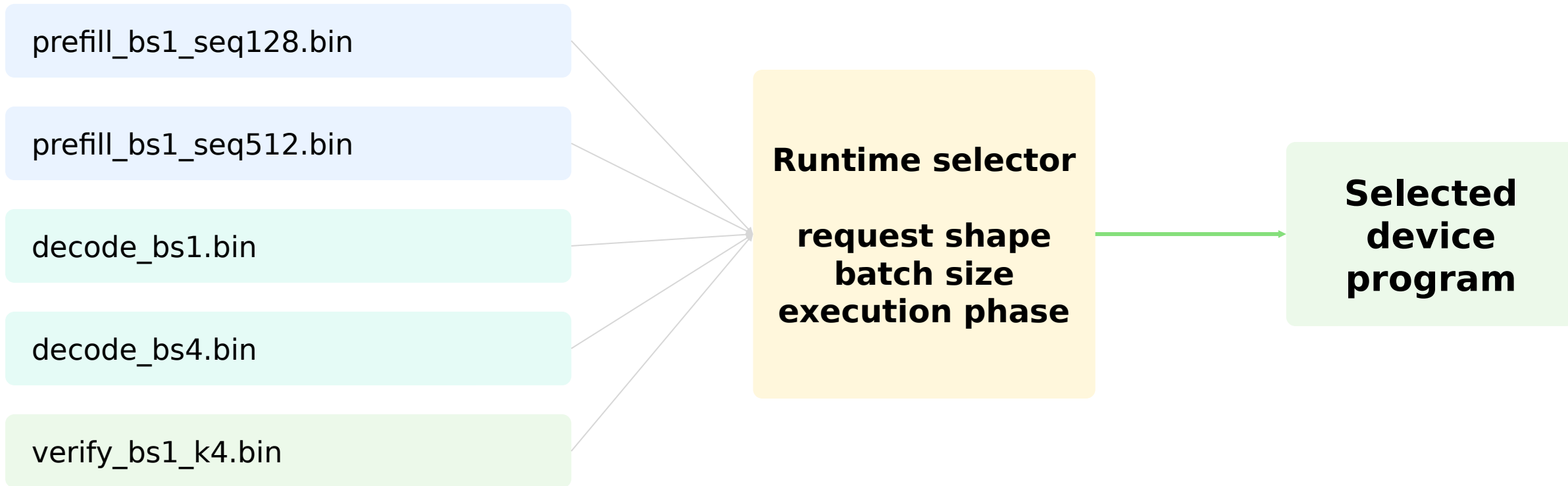
verify_bs1_k4.bin

Runtime selector

**request shape
batch size
execution phase**

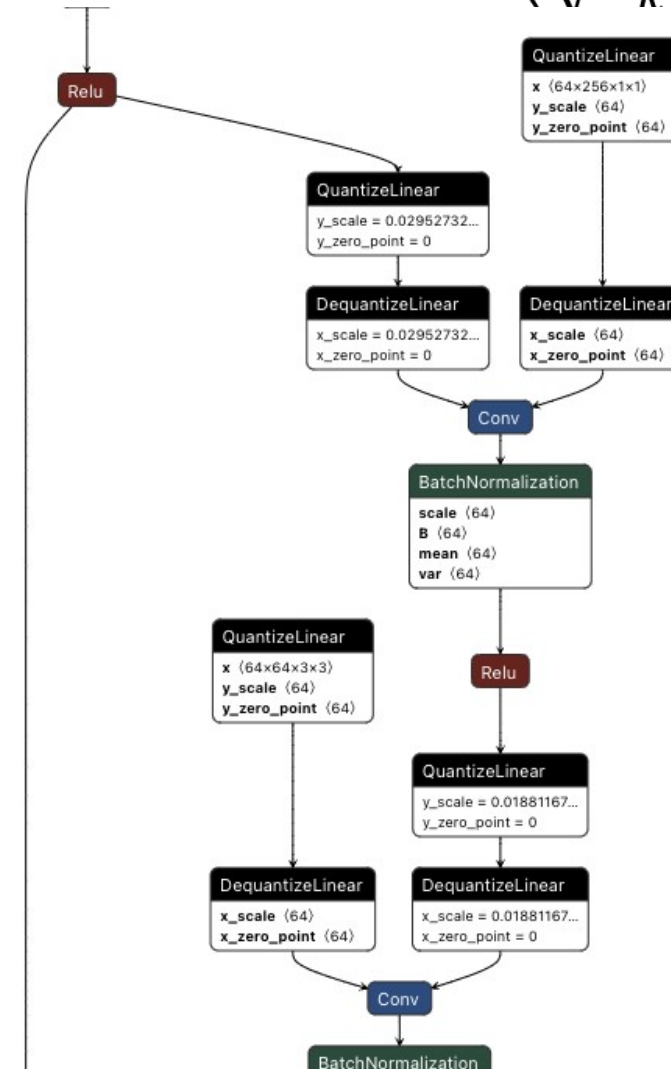
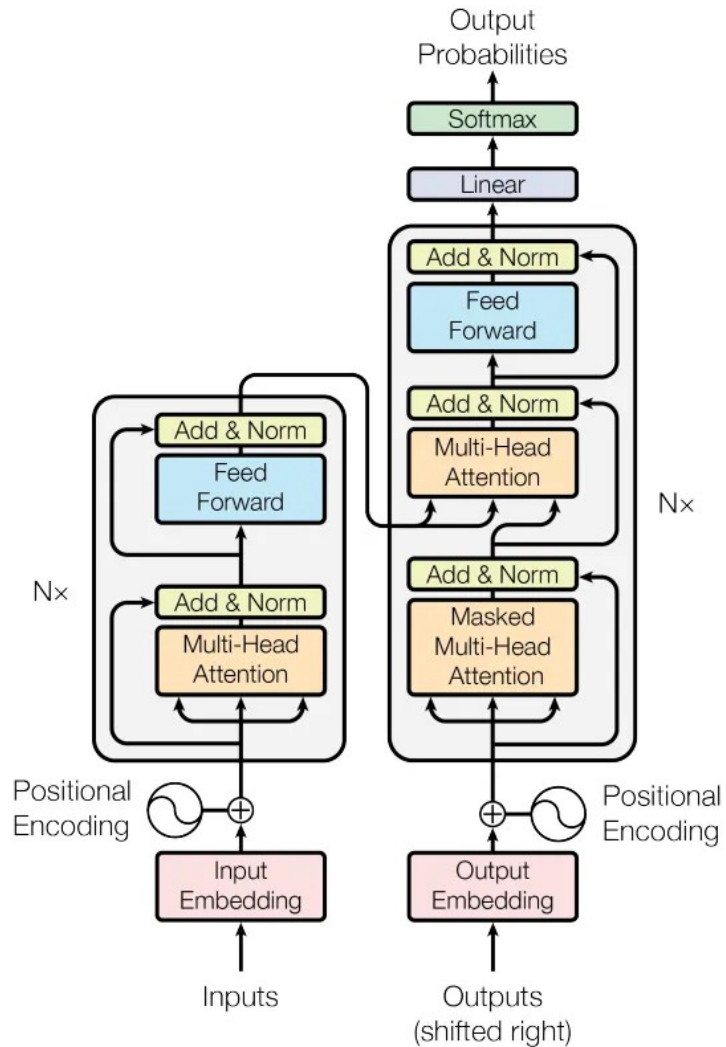
**Selected
device
program**

**More shape buckets improve coverage but increase
compile time and binary size**

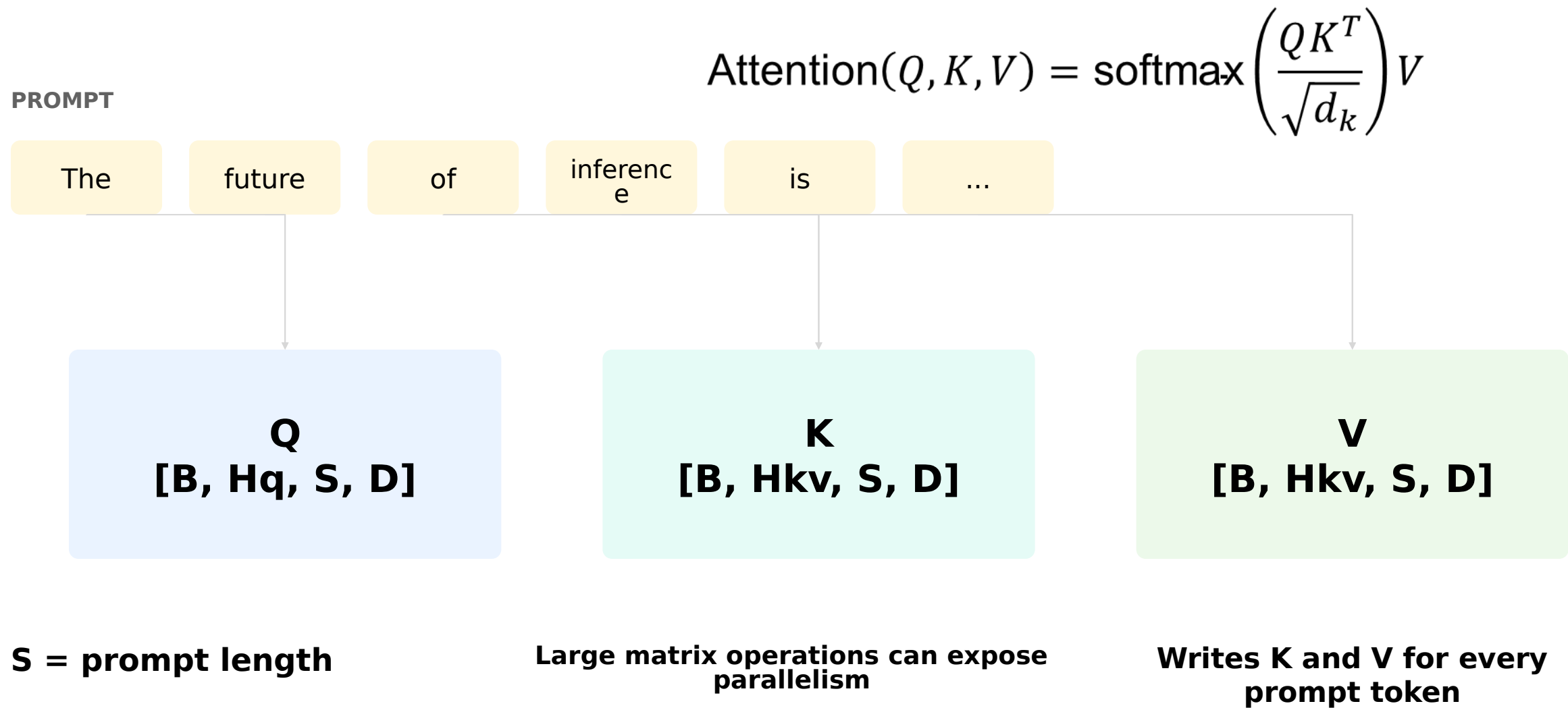


Attention Calculation

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



Prefill Tensor Shapes

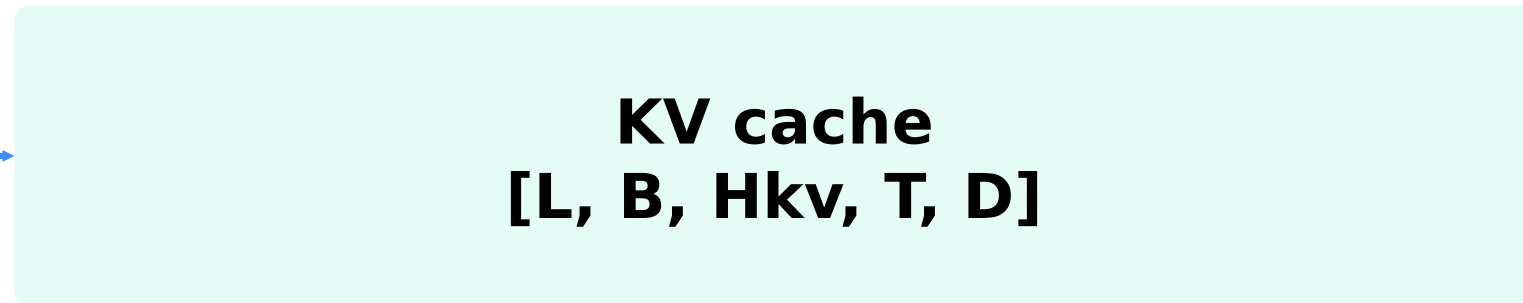
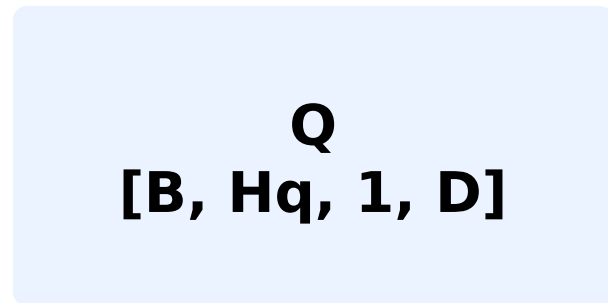


Decode Tensor Shapes

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

ONE NEW QUERY TOKEN

ALL PREVIOUS KEY AND VALUE STATES



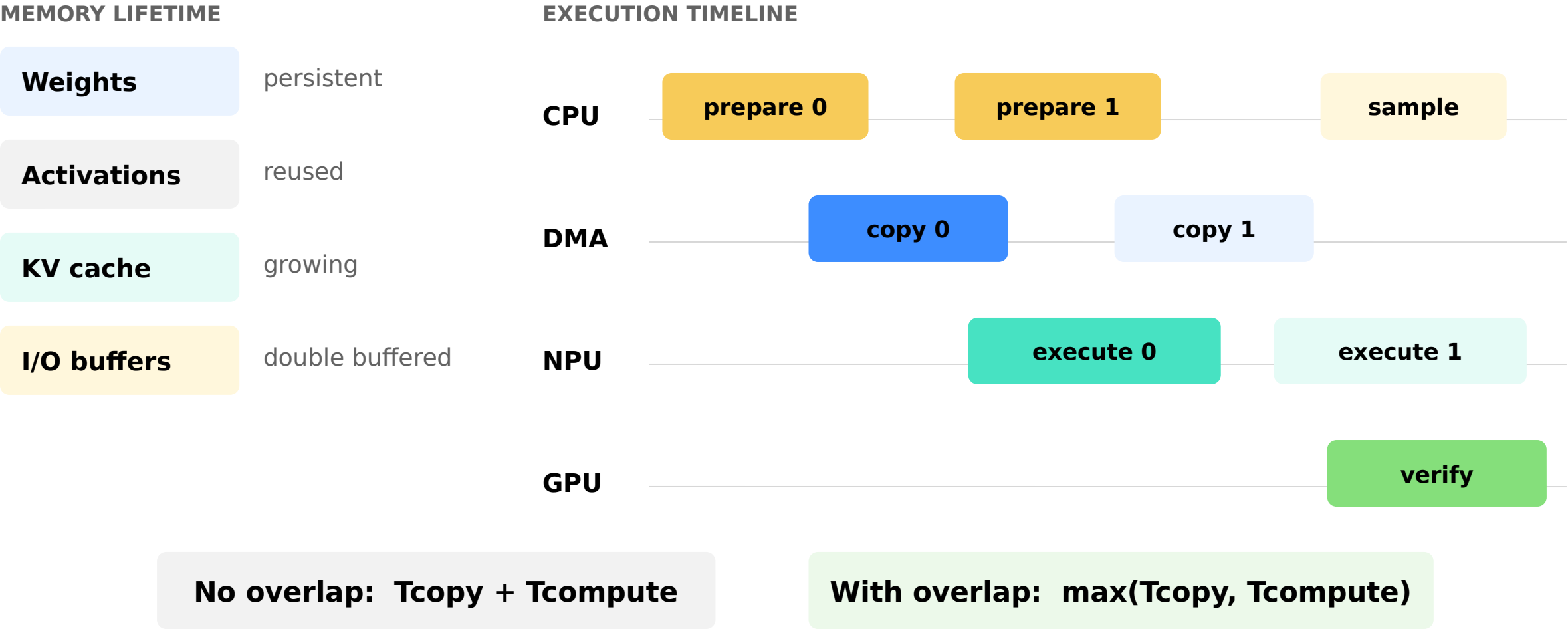
T grows with every generated token

**KV bytes read per
step**

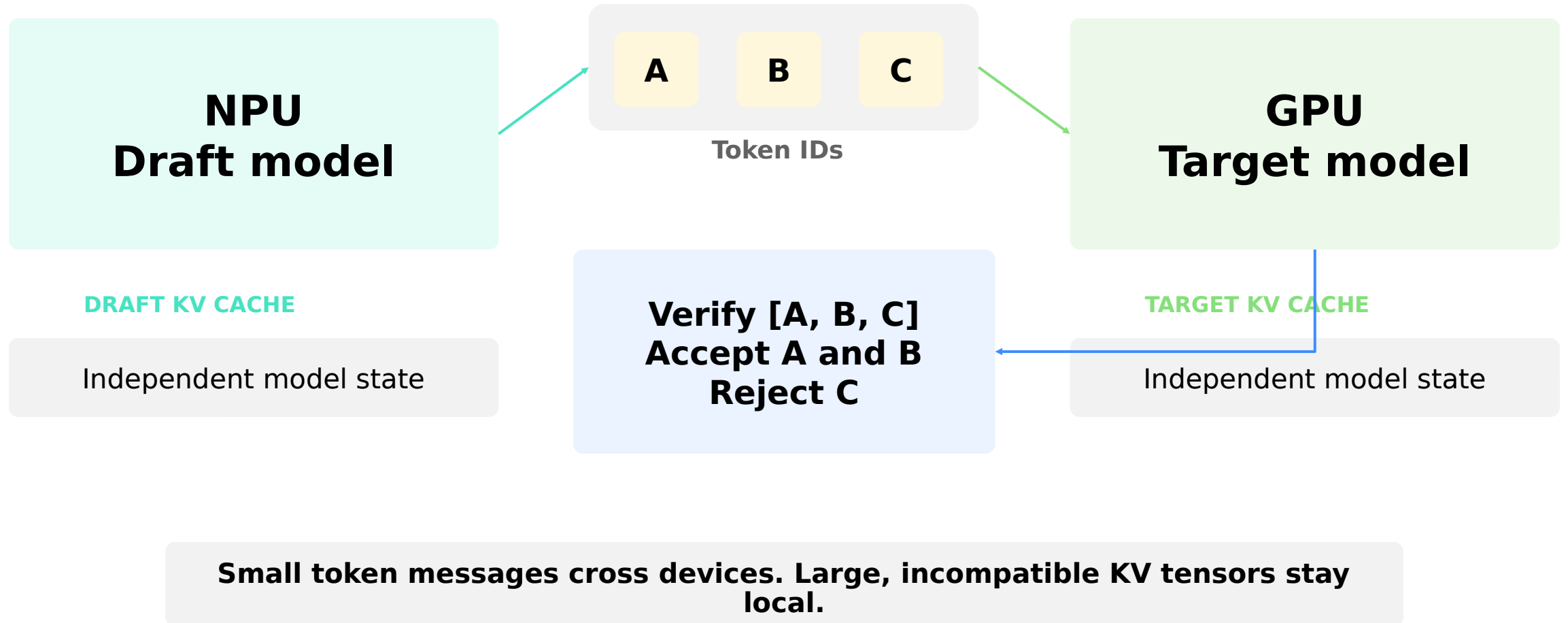
$2 \times L \times B \times Hkv \times T \times D \times \text{dtype size}$

**But in most of the model weight file KV caches are not specified
(Of course, the model weight doesn't know about the device I use)
Engineers need to design it manually**

Runtime Scheduling and Buffer Readiness



Speculative Decoding Across Devices



The Draft Model Must Be Engineered

THE DRAFT PATH

Compatible token space

Small and fast architecture

Supported operators and shapes

Target-specific quantization

High acceptance rate

SPEEDUP CONDITION

$$\mathbf{T_{draft}(K) + T_{transfer} + T_{verify}(K) + T_{sync}}$$

must be lower than

$$\mathbf{K \times T_{target_decode}(1)}$$

Changing layer count or hidden size requires a compatible checkpoint or distillation. Weight reshaping is not enough.

Where Heterogeneous Execution Loses Time

CAUSE	SYSTEM EFFECT
Unsupported operator	CPU fallback and extra copies
Wrong tensor layout	Transpose on the critical path
Late buffer allocation	Device waits for memory
Early synchronization	CPU and devices stop overlapping
Low draft acceptance	Proposed tokens are discarded

Optimize the critical path, not one device benchmark.

Where 42 Skills Fit

42 experience	Inference runtime work
malloc, alignment	Device buffers and tensor arenas, tensor calculations
C++(?)	Most of the runtime API is not fully supported in python yet, nor Rust
Outer Circle - AI/ML Basics	Why not
Outer Circle -- Kernels	Sometimes we need to implement custom kernels(AMD GPU...)
Outer Circle -- Compilers	After making custom kernel, it needs to be interpreted/compiled MILR level differently

Inference engineering is systems programming applied to model execution.

Open Source Entry Points

llama.cpp

C/C++, quantization and hardware backends

[Start here](#)

ONNX Runtime

Graph partitioning and execution providers

[Start here](#)

ExecuTorch

Edge deployment and NPU delegation

[Go deeper](#)

ROCm and HIP

AMD GPU runtime and kernel programming

[Go deeper](#)

vLLM

KV cache, scheduling and speculative decoding

[Serving](#)

<https://github.com/antirez/ds4>

Good first contributions: build fixes, operator tests, benchmarks and backend bug reproduction

Run Open weight model on your phone! (Iphone NPU-GPU-CPU / Qualcomm NPU-CPU)

<https://www.mindstudio.ai/blog/run-gemma-4-locally-google-ai-edge-gallery-phone>

<https://gemma4-ai.com/blog/gemma4-mobile-deploy>

Still not enough people understanding ML/DL deeply and low level programming at the same time
Still (yet so far) vibe codings are not working that well on this area.

Make something run is not a problem, but designing is the main problem

If you're kind of person who's more stick with the old CC, I recommend this area to enter AI

99+ results 📍 Munich (80 km) ▾

How promoted jobs are ranked ?



SW Optimization Engineer AI/ML ✓

Apple
Munich

26 school alumni work here
4 months ago



Machine Learning Compiler Engineer (m/f/d) ✓

Apple
Munich

26 school alumni work here
Viewed · 3 weeks ago



Jr Software Engineer - ML Runtime - Rust

TensorDyne
Munich (Remote)
Saved · 1 month ago



AI Research Engineer - ML Engineering ✓

Helsing
Munich

Viewed · 2 weeks ago



Join our AI Engineering team

Visium
Munich (Hybrid)

Actively reviewing applicants
5 months ago · Easy Apply



Research Engineer, Forge

Mistral
Munich (Hybrid)
3 days ago



Machine Learning Compiler Engineer (m/f/d) ✓

Apple · Munich, Bavaria, Germany (On-site)

Save

Apply



optimizing it for deep learning inference with a focus on performance, scalability, and power efficiency

- Collaborate with cross-functional teams, including hardware and platform architecture teams, to bring new hardware silicon to market and ensure compiler support for next-gen features
- Lead the design and implementation of complex compiler features, advancing both technical capabilities and strategic alignment across the team and company
- Play an instrumental role in defining new compiler architecture approaches and optimizations, balancing trade-offs between performance, energy efficiency, and hardware constraints
- Identify and drive initiatives that will improve the scalability and general performance of AI workloads on Apple hardware, contributing to the vision and roadmap of the Apple Neural Engine team

Minimum Qualifications

- Bachelor's degree in Computer Science, Computer Engineering, or a related field with 3 years of relevant experience
- Experience with program analysis and IR (Intermediate Representation), or programming language design, particularly with MLIR and LLVM
- Proven expertise in compiler design and architecture, including deep experience with front-end and middle-end optimizations, register allocation, and back-end code generation
- High-level proficiency in C++ and experience working with large, complex software systems
- Proficiency in English

Preferred Qualifications

- Master's or PhD degree in Computer Science, Computer Engineering, or a related field
- Demonstrated ability to ship high-quality production software
- Strong communication skills and ability to collaborate effectively across teams and functions
- Experience optimizing compilers for distributed, parallel, or heterogeneous execution environments, with a solid understanding of shared memory, synchronization, and multi-threading techniques
- Expertise in neural network inference on specialized SoCs or GPUs, and knowledge of deep

99+ results 📍 Munich (80 km) ▾

How promoted jobs are ranked ?

 SW Optimization Engineer AI/ML ✓ Apple Munich  26 school alumni work here 4 months ago	✕ Jr Software Engineer - ML Runtime - Rust Tensordyne • Munich, Bavaria, Germany (Remote) Saved Apply  ... programs, because building category-defining technology starts with a healthy, supported team. Come join us as we shape the future of multimodal generative artificial intelligence! About the role As a Software Engineer working within the Low-Level Runtime team, you will build components of an ML model execution framework – low-level assembler architecture hyper-optimized for our hardware – as well as port components of various models to be executed within it. In this role, you will • Maintain and develop low-level ML model execution framework in Rust • Maintain and enhance components and whole models written for our hardware • Optimize performance of ML models, maximizing performance and utilization • Debug and productize models in both virtual and physical hardware environments Preferred qualifications • Experience with Rust programming language • General awareness of high-level ML model concepts: LLMs, attention architectures • Experience with low-level code development – e.g. embedded systems • Experience with driving agents and building LLM engineering workflows Nice to have • Experience with Google Cloud Platform or similar • Experience with Python • Deep knowledge of modern LLM attention architectures (GDN, DSA, KDA etc)
 Machine Learning Compiler Engineer (m/f/d) ✓ Apple Munich  26 school alumni work here Viewed · 3 weeks ago	
 Jr Software Engineer - ML Runtime - Rust Tensordyne Munich (Remote) Saved · 1 month ago	
 Join our AI Engineering team Visium Munich (Hybrid)  Actively reviewing applicants 5 months ago ·  Easy Apply	
 Research Engineer, Forge Mistral Munich (Hybrid) 3 days ago	
 AI Software Engineer Hyra Munich (Hybrid)	

**SW Optimization Engineer AI/ML** ✓Apple
Munich

26 school alumni work here

4 months ago

**Machine Learning Compiler Engineer (m/f/d)** ✓Apple
Munich

26 school alumni work here

Viewed · 3 weeks ago

**Jr Software Engineer - ML Runtime - Rust**Tensordyne
Munich (Remote)
Saved · 1 month ago**Join our AI Engineering team**Visium
Munich (Hybrid)

Actively reviewing applicants

5 months ago · Easy Apply

**Research Engineer, Forge**Mistral
Munich (Hybrid)
3 days ago**AI Software Engineer**Hyra
Munich (Hybrid)**Jr Software Engineer - ML Runtime - Rust**

Tensordyne · Munich, Bavaria, Germany (Remote)

Saved

Apply



programs, because building category-defining technology starts with a healthy, supported team. Come join us as we shape the future of multimodal generative artificial intelligence!

About the role

As a Software Engineer working within the Low-Level Runtime team, you will build components of an ML model execution framework – low-level assembler architecture hyper-optimized for our hardware – as well as port components of various models to be executed within it.

In this role, you will

- Maintain and develop low-level ML model execution framework in Rust
- Maintain and enhance components and whole models written for our hardware
- Optimize performance of ML models, maximizing performance and utilization
- Debug and productize models in both virtual and physical hardware environments

Preferred qualifications

- Experience with Rust programming language
- General awareness of high-level ML model concepts: LLMs, attention architectures
- Experience with low-level code development – e.g. embedded systems
- Experience with driving agents and building LLM engineering workflows

Nice to have

- Experience with Google Cloud Platform or similar
- Experience with Python
- Deep knowledge of modern LLM attention architectures (GDN, DSA, KDA etc)

Q&A